



ECEN 5793

Digital Image Processing

Nate Lannan

Agenda

Coding Examples

Image Registration

Questions

How to take advantage of array operations for efficient image processing operations?

Zooming and Shrinking

Both zooming and shrinking are applied to digital images with two steps

- The creation of new pixel locations
- The assignment of gray levels to those new locations

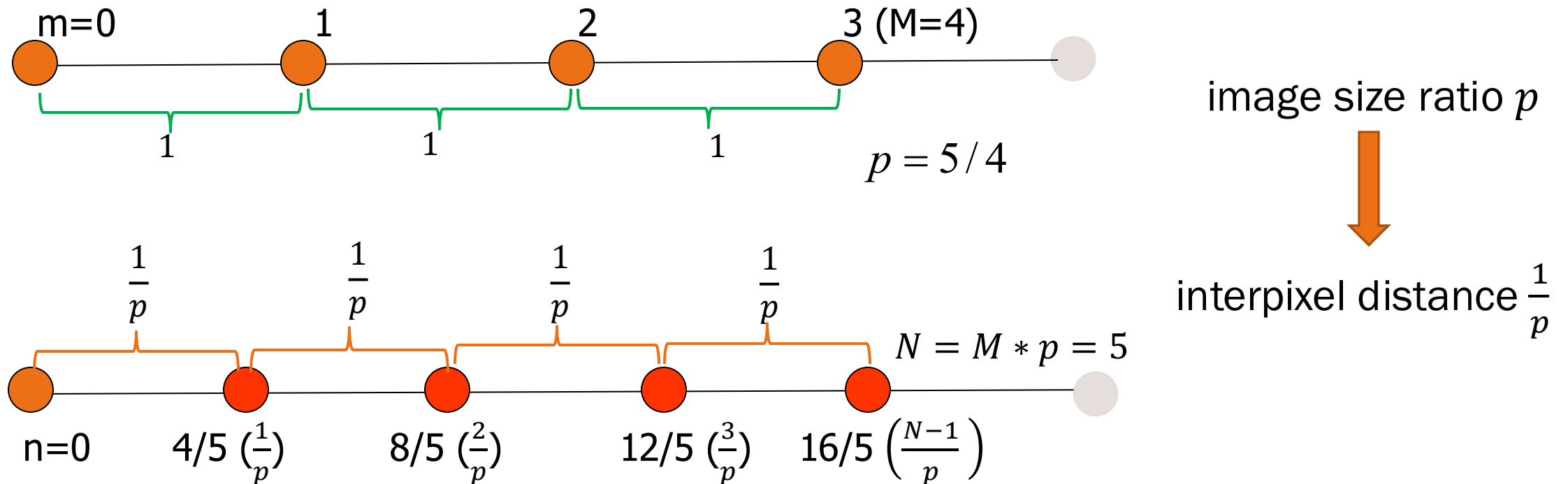
Pixel Interpolation is the process of using known data to estimate values at unknown locations.

- Nearest neighbor interpolation (ZOH)
- Pixel replication (special case of nearest neighbor interpolation)
- Bilinear interpolation
- Bicubic interpolation

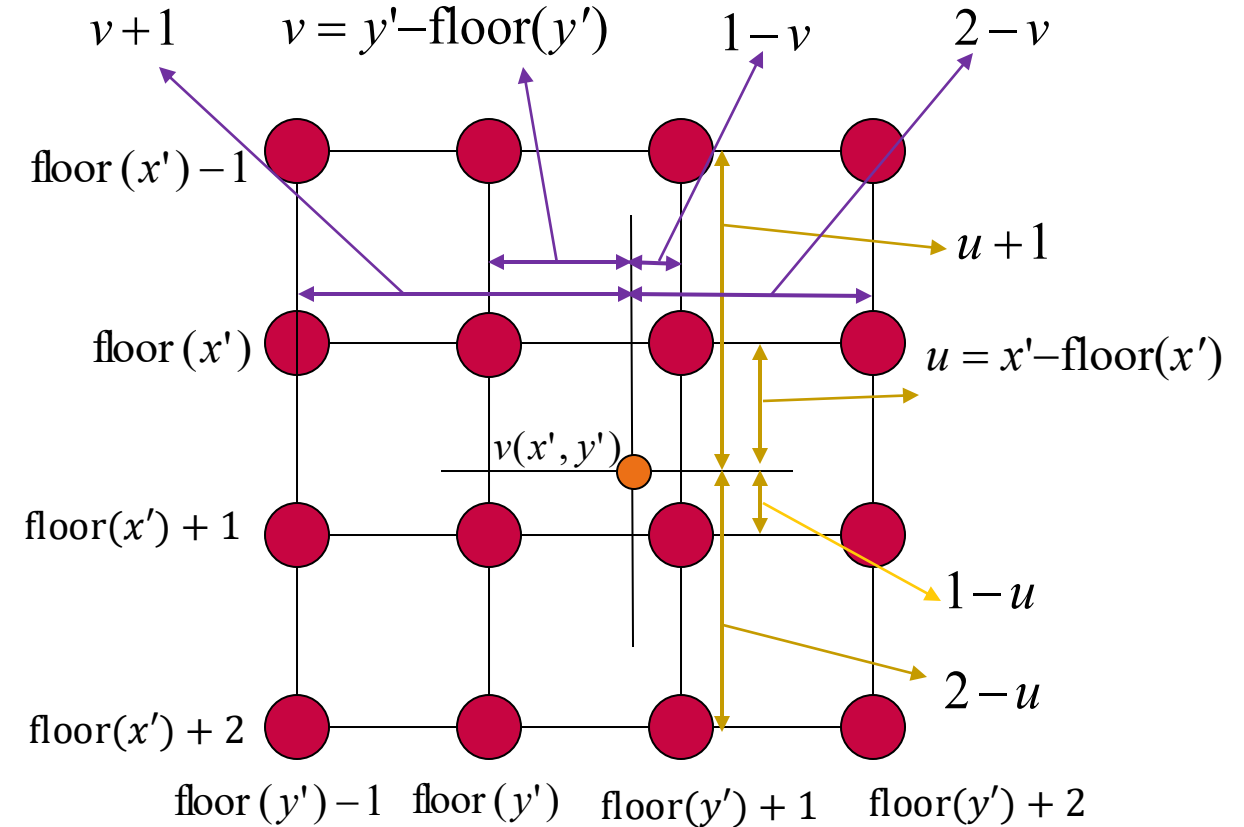
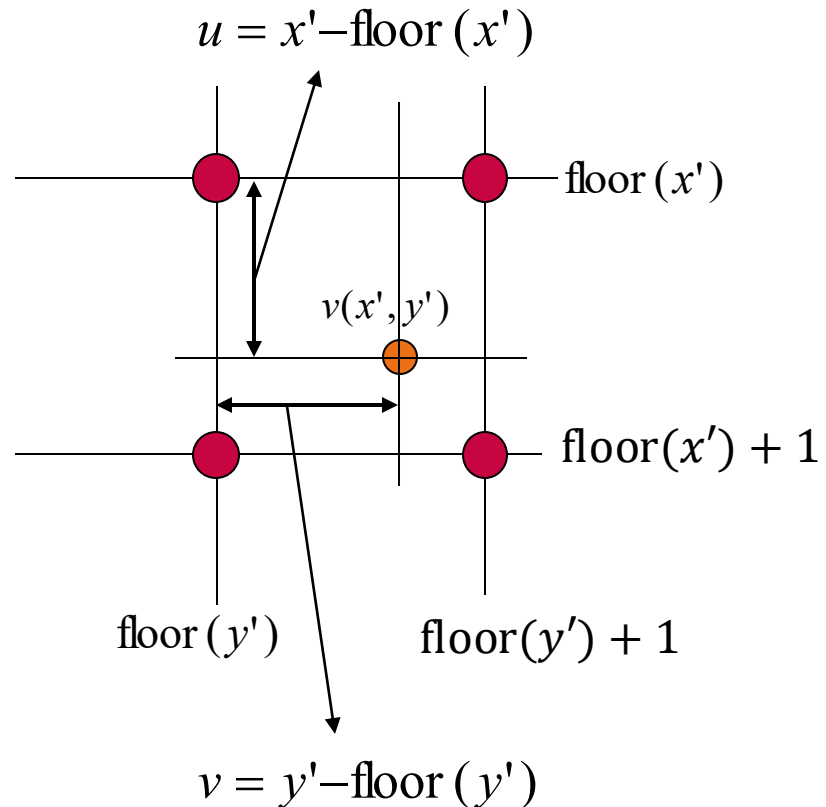
Zooming and shrinking can be done in a similar manner.

How to create new pixel locations?

In the zooming case, when the image size ratio p is not integer, usually new pixels (red points) are created not on the grid.



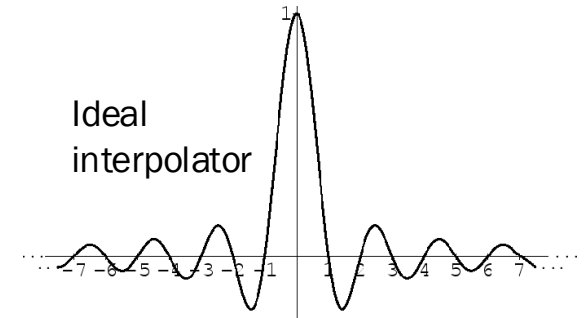
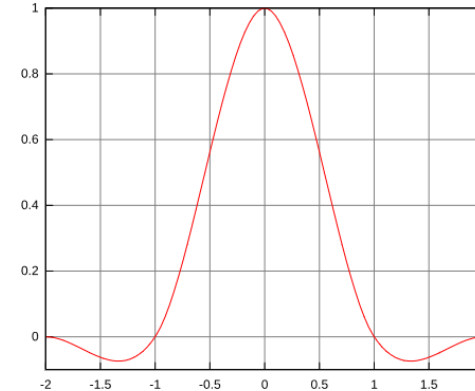
Linear vs. Bicubic Interpolation



Bicubic Interpolation (Cont'd)

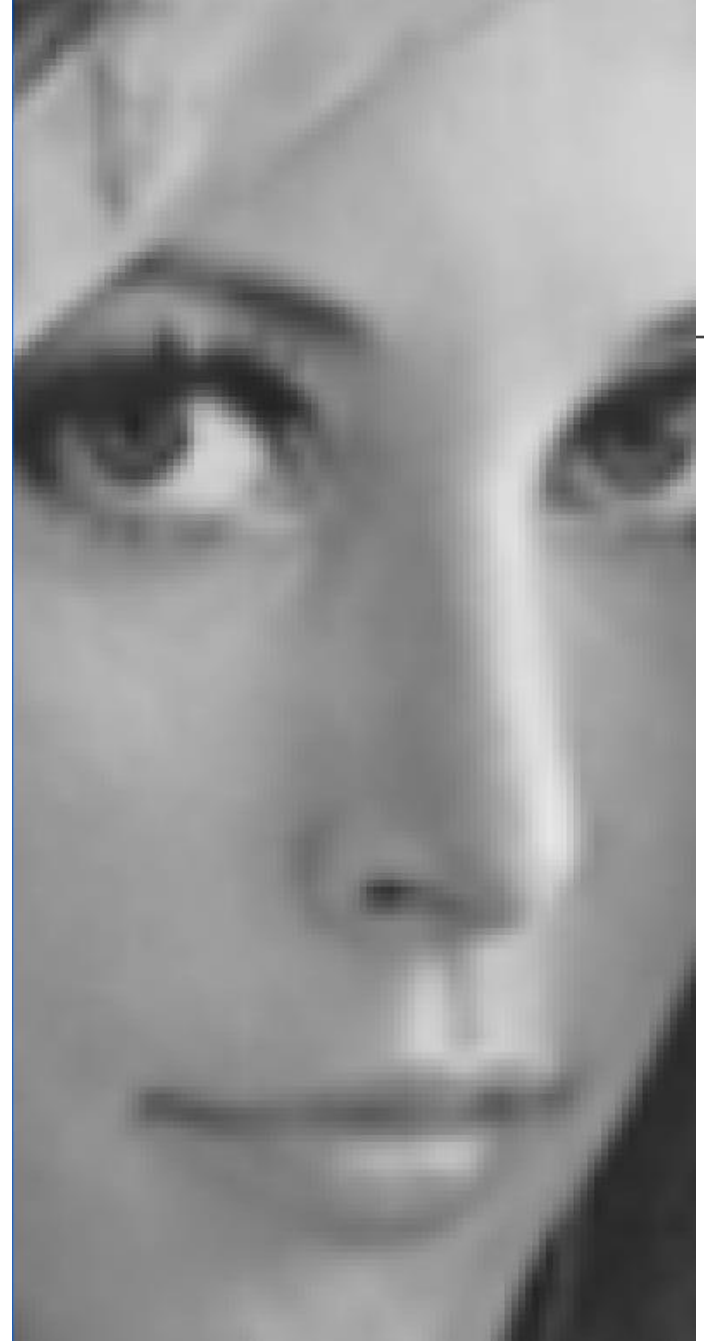
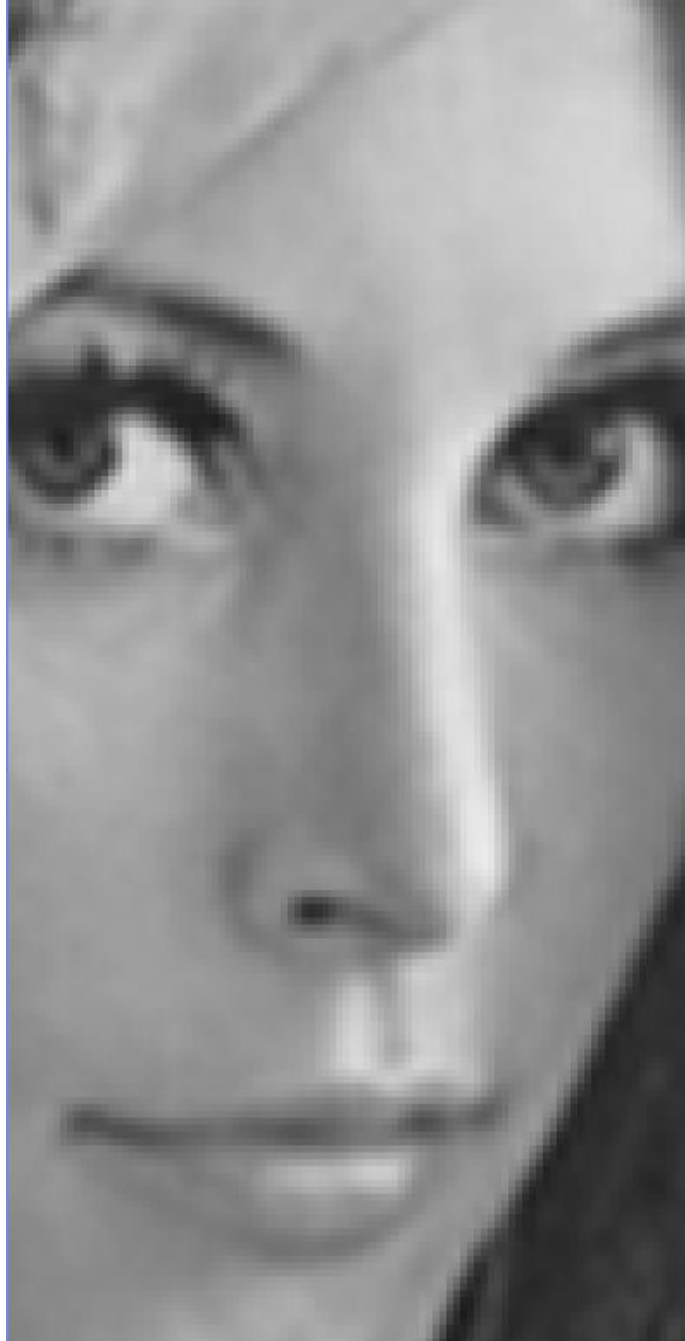
The interpolation kernel using the Ricker wavelet is given by

$$h(x) = \begin{cases} 1 - 2|x|^2 + |x|^3 & \text{for } 0 \leq |x| \leq 1 \\ 4 - 8|x| + 5|x|^2 - |x|^3 & \text{for } 1 < |x| \leq 2 \\ 0 & \text{otherwise} \end{cases}$$



Then the 2D bicubic interpolation is computed as

$$v(x', y') = \sum_{m=1}^4 \sum_{n=1}^4 h(x_m - x') h(y_n - y') f(x_m, y_n)$$



MATLAB Programming

Generating New Domain

First, given the original image size N and a zoom ratio p , we need find the new image size $M = \text{round}(N * p)$.

Then generate a new domain $X = 1:\frac{1}{p}:(\frac{M-1}{p} + 1)$ that should have M samples from 1 to N with interval $\frac{1}{p}$. (Example: $N=10$, $p=4.5$, then $M=45$)

```
>> X=1:1/4.5:(44/4.5+1)
```

```
X =
```

```
Columns 1 through 15
```

```
1.0000    1.2222    1.4444    1.6667    1.8889    2.1111    2.3333    2.5556    2.7778    3.0000    3.2222    3.4444    3.6667    3.8889    4.1111
```

```
Columns 16 through 30
```

```
4.3333    4.5556    4.7778    5.0000    5.2222    5.4444    5.6667    5.8889    6.1111    6.3333    6.5556    6.7778    7.0000    7.2222    7.4444
```

```
Columns 31 through 45
```

```
7.6667    7.8889    8.1111    8.3333    8.5556    8.7778    9.0000    9.2222    9.4444    9.6667    9.8889   10.1111   10.3333   10.5556   10.7778
```

Generating New Domain In Python

In python this is made much simpler through the fact that we start counting at 0 and we have a function to yield all integers from a start point to an end point. `x=np.arange(M)/p`

(Example: $N=10$, $p=4.5$, then $M=45$)

```
>>> import numpy as np
>>> x = np.arange(45) / 4.5
>>> x
array([0.          , 0.22222222, 0.44444444, 0.66666667, 0.88888889,
       1.11111111, 1.33333333, 1.55555556, 1.77777778, 2.          ,
       2.22222222, 2.44444444, 2.66666667, 2.88888889, 3.11111111,
       3.33333333, 3.55555556, 3.77777778, 4.          , 4.22222222,
       4.44444444, 4.66666667, 4.88888889, 5.11111111, 5.33333333,
       5.55555556, 5.77777778, 6.          , 6.22222222, 6.44444444,
       6.66666667, 6.88888889, 7.11111111, 7.33333333, 7.55555556,
       7.77777778, 8.          , 8.22222222, 8.44444444, 8.66666667,
       8.88888889, 9.11111111, 9.33333333, 9.55555556, 9.77777778])
```

Matlab Programming

Nearest Neighbor Interpolation



```
clear all; p=4;  
I=imread('lena128.bmp');  
[x,y] = size(I);  
X=round(x*p); Y=round(y*p);  
u=1:1/p:((X-1)/p+1); v=1:1/p:((Y-1)/p+1);  
U=round(u); V=round(v);  
U(find(U>x))=x; V(find(V>y))=y;  
A = I(U,V);  
figure(1);imshow(I);  
figure(2);imshow(A);
```



Python Programming

Nearest Neighbor Interpolation



```
import numpy as np
import matplotlib.pyplot as plt
from imageio import imread
```

```
p = 4
I = imread('lena128.bmp')
```

```
x, y = I.shape
X = round(x * p)
Y = round(y * p)
```

```
u = np.arange(X)/p
v = np.arange(Y)/p
```

```
U = np.round(u).astype(int)
V = np.round(v).astype(int)
```

```
U[U > x-1] = x-1
V[V > y-1] = y-1
A = I[np.ix_(U, V)]
```

```
plt.figure(1)
plt.imshow(I, cmap='gray')
plt.title('Original')
```

```
plt.figure(2)
plt.imshow(A, cmap='gray')
plt.title('Upsampled (nearest neighbor)')
```

```
plt.show()
```



Python Programming Notes

- `plt.show()` is necessary to render your image to the screen. `matplotlib` builds an in-memory scene graph that it does not hand off to the rendering back end (Tk, Qt, GTK, etc.) until the `show` function is called.
- `np.ix_()` is a fancy-indexing helper for selecting rows and columns from an array without triggering NumPy's pairing behavior.

```
>>> A = np.arange(16).reshape(4,4)
>>> A
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [12, 13, 14, 15]])
>>> rows = [0,2]
>>> columns = [1,3]
>>> foo = A[rows,columns]
>>> foo
array([ 1, 11])
```

```
>>> A = np.arange(16).reshape(4,4)
>>> A
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [12, 13, 14, 15]])
>>> rows = [0,2]
>>> columns = [1,3]
>>> foo = A[np.ix_(rows,columns)]
>>> foo
array([[ 1,  3],
       [ 9, 11]])
```

Python Programming Notes

Once you have installed python you can use it in two ways (much like MATLAB), in an interactive cmd prompt environment, or as an interpreted programming language. Install it and call it from your favorite cmd line environment either without an argument (cmd line) or with a .py file argument to run a script.

- <https://learn.microsoft.com/en-us/windows/terminal/install>
- <https://learn.microsoft.com/en-us/windows/wsl/install>

```
C:\Users\amerigo\Documents\ECEN5793\ECEN5793\Projects\Project1>python
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

```
C:\Users\amerigo\Documents\ECEN5793\ECEN5793\Projects\Project1>python testRicker.py
C:\Users\amerigo\Documents\ECEN5793\ECEN5793\Projects\Project1>
```

Python Programming Notes

```
import numpy as np
import matplotlib.pyplot as plt
from imageio import imread
import pdb

p = 4
I = imread('lena128.bmp')

x, y = I.shape

X = round(x * p)
Y = round(y * p)

pdb.set_trace()
u = np.arange(X)/p
v = np.arange(Y)/p

U = np.round(u).astype(int)
V = np.round(v).astype(int)

U[U > x-1] = x-1
V[V > y-1] = y-1

A = I[np.ix_(U, V)]

plt.figure(1)
plt.imshow(I, cmap='gray')
plt.title('Original')

plt.figure(2)
plt.imshow(A, cmap='gray')
plt.title('Upsampled (nearest neighbor)')

plt.show()
```

A huge help is the python debugger (pdb). This will allow you to set breakpoints, step through code, and investigate values in memory.

<https://docs.python.org/3/library/pdb.html>

```
> c:\users\amerigo\documents\ecen5793\ecen5793\projects\project1\example1.py(14)<module>()
-> pdb.set_trace()
(Pdb) n
> c:\users\amerigo\documents\ecen5793\ecen5793\projects\project1\example1.py(15)<module>()
-> u = np.arange(X)/p
(Pdb) n
> c:\users\amerigo\documents\ecen5793\ecen5793\projects\project1\example1.py(16)<module>()
-> v = np.arange(Y)/p
(Pdb) n
> c:\users\amerigo\documents\ecen5793\ecen5793\projects\project1\example1.py(18)<module>()
-> U = np.round(u).astype(int)
(Pdb) v[0:20]
array([0.  , 0.25, 0.5  , 0.75, 1.  , 1.25, 1.5  , 1.75, 2.  , 2.25, 2.5  ,
       2.75, 3.  , 3.25, 3.5  , 3.75, 4.  , 4.25, 4.5  , 4.75])
(Pdb) |
```

Matlab Programming

From 1D Domain to 2D Grid

Function `[X1,X2,X3,...] = NDGRID(x1,x2,x3,...)` transforms the domain specified by 1d arrays `x1,x2,x3`, into Nd arrays `X1, X2, X3,...`

$$x1 = [1 \ 3 \ 5 \ 7]$$

$$x2 = [2 \ 4 \ 6]$$

$$[X1,X2] = \text{NDGRID}(x1,x2)$$

$$X1 = \begin{bmatrix} 1 & 1 & 1 \\ 3 & 3 & 3 \\ 5 & 5 & 5 \\ 7 & 7 & 7 \end{bmatrix}_{4 \times 3}$$

$$X2 = \begin{bmatrix} 2 & 4 & 6 \\ 2 & 4 & 6 \\ 2 & 4 & 6 \\ 2 & 4 & 6 \end{bmatrix}_{4 \times 3}$$

```
>> [X1,X2]=ndgrid(x1,x2);
```

```
>> X1
```

X1 =

1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
1.3000	1.3000	1.3000	1.3000	1.3000	1.3000
1.6000	1.6000	1.6000	1.6000	1.6000	1.6000
1.9000	1.9000	1.9000	1.9000	1.9000	1.9000
2.2000	2.2000	2.2000	2.2000	2.2000	2.2000
2.5000	2.5000	2.5000	2.5000	2.5000	2.5000
2.8000	2.8000	2.8000	2.8000	2.8000	2.8000

```
>> X2
```

X2 =

1.0000	1.7000	2.4000	3.1000	3.8000	4.5000
1.0000	1.7000	2.4000	3.1000	3.8000	4.5000
1.0000	1.7000	2.4000	3.1000	3.8000	4.5000
1.0000	1.7000	2.4000	3.1000	3.8000	4.5000
1.0000	1.7000	2.4000	3.1000	3.8000	4.5000
1.0000	1.7000	2.4000	3.1000	3.8000	4.5000
1.0000	1.7000	2.4000	3.1000	3.8000	4.5000

x1 =

1.0000	1.3000	1.6000	1.9000	2.2000	2.5000	2.8000
--------	--------	--------	--------	--------	--------	--------

x2 =

1.0000	1.7000	2.4000	3.1000	3.8000	4.5000
--------	--------	--------	--------	--------	--------

Python Programming

From 1D Domain to 2D Grid

Function `X1,X2,X3,... = np.meshgrid(x1,x2,x3,..., indexing='ij')` transforms the domain specified by 1d arrays `x1,x2,x3`, into Nd arrays `X1, X2, X3,...`

```
>>> x1 = np.array([1,3,5,7])
>>> x2 = np.array([2,4,6])
>>> X1, X2 = np.meshgrid(x1, x2, indexing='ij')
>>> X1
array([[1, 1, 1],
       [3, 3, 3],
       [5, 5, 5],
       [7, 7, 7]])
>>> X2
array([[2, 4, 6],
       [2, 4, 6],
       [2, 4, 6],
       [2, 4, 6]])
```

Matlab Programming

Array operation between two arrays of the same size

Array operations between two arrays of the same size

- Multiplication: “.*”
- Subtraction: “.-”
- Addition: “.+”

```
>> X3=X1.*X2
```

X3 =

1.0000	1.7000	2.4000	3.1000	3.8000	4.5000
1.3000	2.2100	3.1200	4.0300	4.9400	5.8500
1.6000	2.7200	3.8400	4.9600	6.0800	7.2000
1.9000	3.2300	4.5600	5.8900	7.2200	8.5500
2.2000	3.7400	5.2800	6.8200	8.3600	9.9000
2.5000	4.2500	6.0000	7.7500	9.5000	11.2500
2.8000	4.7600	6.7200	8.6800	10.6400	12.6000

```
>> [X1,X2]=ndgrid(x1,x2);
```

```
>> X1
```

X1 =

1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
1.3000	1.3000	1.3000	1.3000	1.3000	1.3000
1.6000	1.6000	1.6000	1.6000	1.6000	1.6000
1.9000	1.9000	1.9000	1.9000	1.9000	1.9000
2.2000	2.2000	2.2000	2.2000	2.2000	2.2000
2.5000	2.5000	2.5000	2.5000	2.5000	2.5000
2.8000	2.8000	2.8000	2.8000	2.8000	2.8000

```
>> X2
```

X2 =

1.0000	1.7000	2.4000	3.1000	3.8000	4.5000
1.0000	1.7000	2.4000	3.1000	3.8000	4.5000
1.0000	1.7000	2.4000	3.1000	3.8000	4.5000
1.0000	1.7000	2.4000	3.1000	3.8000	4.5000
1.0000	1.7000	2.4000	3.1000	3.8000	4.5000
1.0000	1.7000	2.4000	3.1000	3.8000	4.5000
1.0000	1.7000	2.4000	3.1000	3.8000	4.5000

Python Programming

Array operation between two arrays of the same size

numpy operations are elementwise
by default

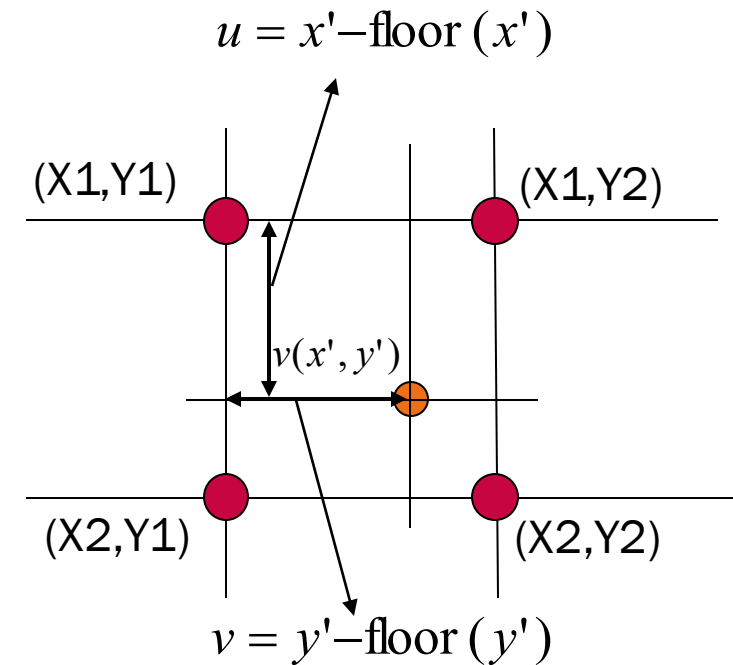
- Multiplication: “*”
- Subtraction: “-”
- Addition: “+”

```
>>> X1,X2 =np.meshgrid(x1,x2,indexing='ij')
>>> X3=X1*X2
>>> X1
array([[1. , 1. , 1. , 1. , 1. , 1. ],
       [1.3, 1.3, 1.3, 1.3, 1.3, 1.3],
       [1.6, 1.6, 1.6, 1.6, 1.6, 1.6],
       [1.9, 1.9, 1.9, 1.9, 1.9, 1.9],
       [2.2, 2.2, 2.2, 2.2, 2.2, 2.2],
       [2.5, 2.5, 2.5, 2.5, 2.5, 2.5],
       [2.8, 2.8, 2.8, 2.8, 2.8, 2.8]])
>>> X2
array([[1. , 1.7, 2.4, 3.1, 3.8, 4.5],
       [1. , 1.7, 2.4, 3.1, 3.8, 4.5],
       [1. , 1.7, 2.4, 3.1, 3.8, 4.5],
       [1. , 1.7, 2.4, 3.1, 3.8, 4.5],
       [1. , 1.7, 2.4, 3.1, 3.8, 4.5],
       [1. , 1.7, 2.4, 3.1, 3.8, 4.5],
       [1. , 1.7, 2.4, 3.1, 3.8, 4.5]])
>>> X3
array([[ 1. ,  1.7 ,  2.4 ,  3.1 ,  3.8 ,  4.5 ],
       [ 1.3 ,  2.21,  3.12,  4.03,  4.94,  5.85],
       [ 1.6 ,  2.72,  3.84,  4.96,  6.08,  7.2 ],
       [ 1.9 ,  3.23,  4.56,  5.89,  7.22,  8.55],
       [ 2.2 ,  3.74,  5.28,  6.82,  8.36,  9.9 ],
       [ 2.5 ,  4.25,  6. ,  7.75,  9.5 , 11.25],
       [ 2.8 ,  4.76,  6.72,  8.68, 10.64, 12.6 ]])
```

Matlab Programming

Image Bilinear Interpolation

```
p=4;
I=imread('lena128.bmp'); I=im2double(I);
[x,y] = size(I); X=round(x*p); Y=round(y*p);
u=1:1/p:((X-1)/p+1); v=1:1/p:((Y-1)/p+1);
[XI,YI]=ndgrid(u,v);
UI=XI-floor(XI); VI=YI-floor(YI);
X1=floor(u); Y1=floor(v); X2=floor(u)+1; Y2=floor(v)+1;
X2(find(X2>x))=x; Y2(find(Y2>y))=y;
I1=I(X1,Y1); I2=I(X1,Y2); I3=I(X2,Y1); I4=I(X2,Y2);
c1=(1-UI).*(1-VI); c2=(1-UI).*VI; c3=UI.*(1-VI); c4=UI.*VI;
B=c1.*I1+c2.*I2+c3.*I3+c4.*I4;
imshow(B);
```

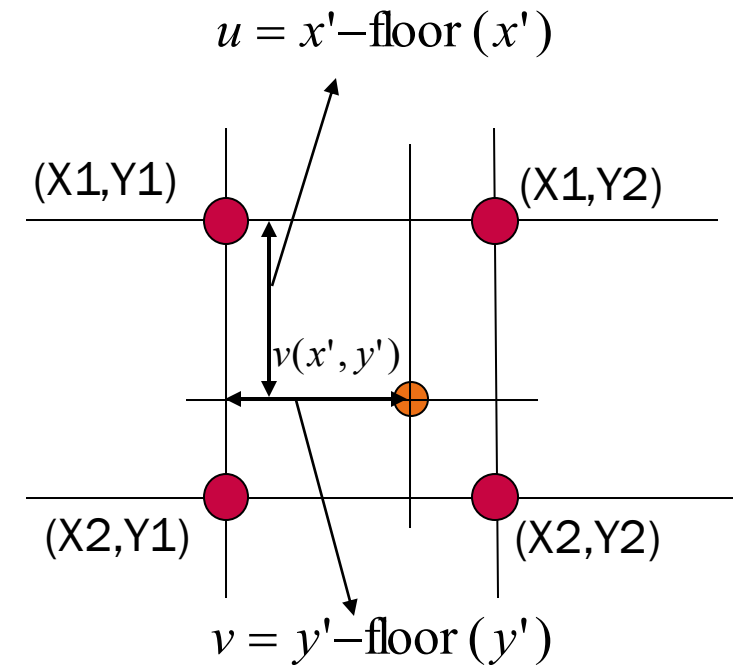


Python Programming

Image Bilinear Interpolation

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from imageio import imread
4 import pdb
5
6 p = 4
7
8 II = imread('lena128.bmp')
9 if II.ndim == 3:
10     II = II[..., 0]
11 I = II.astype(float) / 255.0
12
13 x, y = I.shape
14 X = round(x * p)
15 Y = round(y * p)
16
17 u = np.arange(X)/p
18 v = np.arange(Y)/p
19
20 XI, YI = np.meshgrid(u, v, indexing='ij')
21
22 UI = XI - np.floor(XI)
23 VI = YI - np.floor(YI)
24
25 # --- floor indices
26 X1 = np.floor(u).astype(int)
27 Y1 = np.floor(v).astype(int)
28 X2 = X1 + 1
29 Y2 = Y1 + 1
```

```
30
31 # clamp to image size
32 X2[X2 >= x] = x - 1
33 Y2[Y2 >= y] = y - 1
34
35 # --- sample the four neighbors
36 I1 = I[np.ix_(X1, Y1)]
37 I2 = I[np.ix_(X1, Y2)]
38 I3 = I[np.ix_(X2, Y1)]
39 I4 = I[np.ix_(X2, Y2)]
40
41 # --- bilinear weights
42 c1 = (1 - UI) * (1 - VI)
43 c2 = (1 - UI) * VI
44 c3 = UI * (1 - VI)
45 c4 = UI * VI
46
47 # --- interpolation
48 B = c1 * I1 + c2 * I2 + c3 * I3 + c4 * I4
49
50 plt.imshow(B, cmap='gray')
51 plt.title('Bilinear upsample')
52 plt.axis('off')
53 plt.show()
```





Questions

How can we estimate the geometric transformation function and use it to register the two images?



Input image
from Camera #1

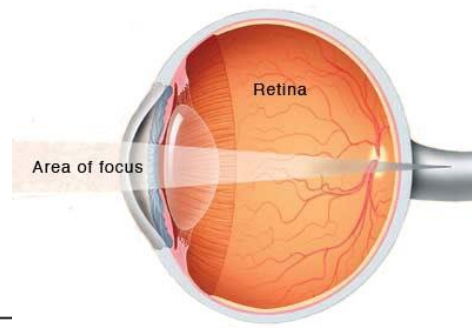


Reference image
from Camera #2

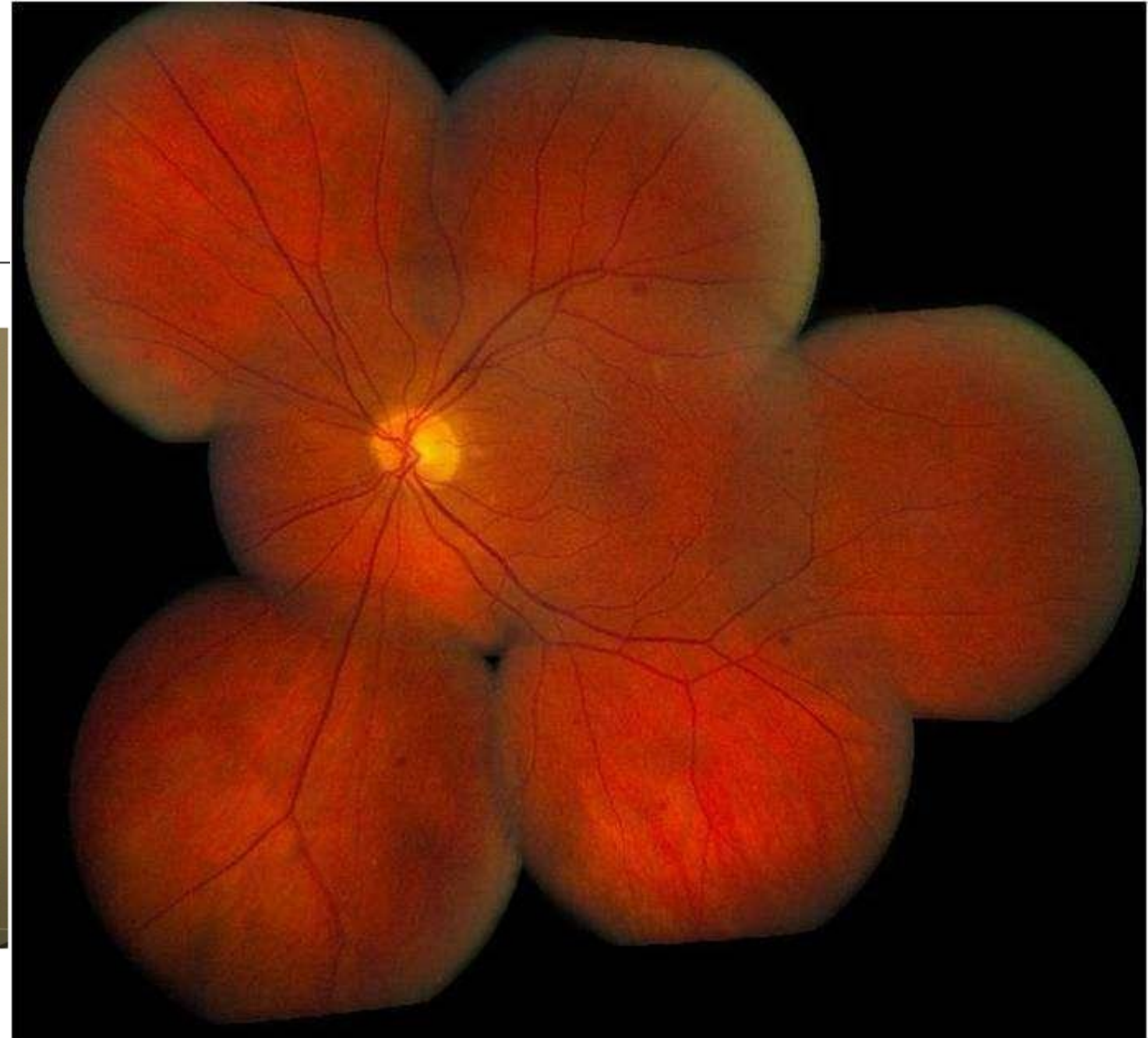


<http://www.mathworks.com/discovery/image-registration.html>

Retinal Image Registration



© MAYO FOUNDATION FOR MEDICAL EDUCATION AND RESEARCH. ALL RIGHTS RESERVED.



<http://www.rsipvision.com/portfolio/image-stitching-of-the-retina/>

Geometric Spatial Transformation from an Input Image to the Reference Image

There are two steps involved a geometric transformation

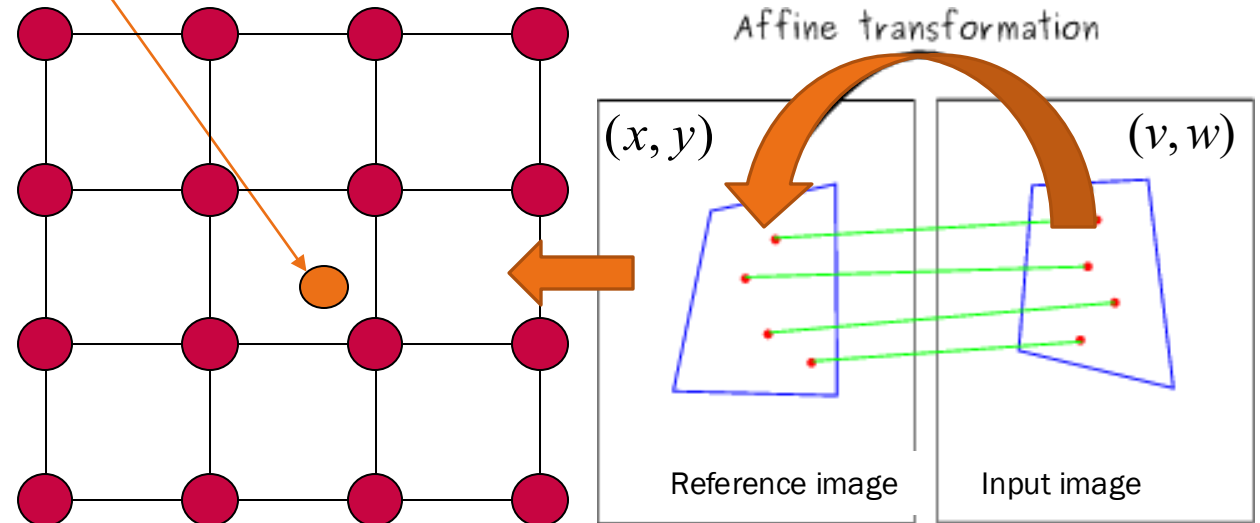
- A spatial transformation of coordinates from an input to the reference image

$$(x, y) = \mathbf{T}\{(v, w)\}$$

- where (v, w) are pixel coordinates in an input image and (x, y) are the corresponding pixel coordinates in the reference image.

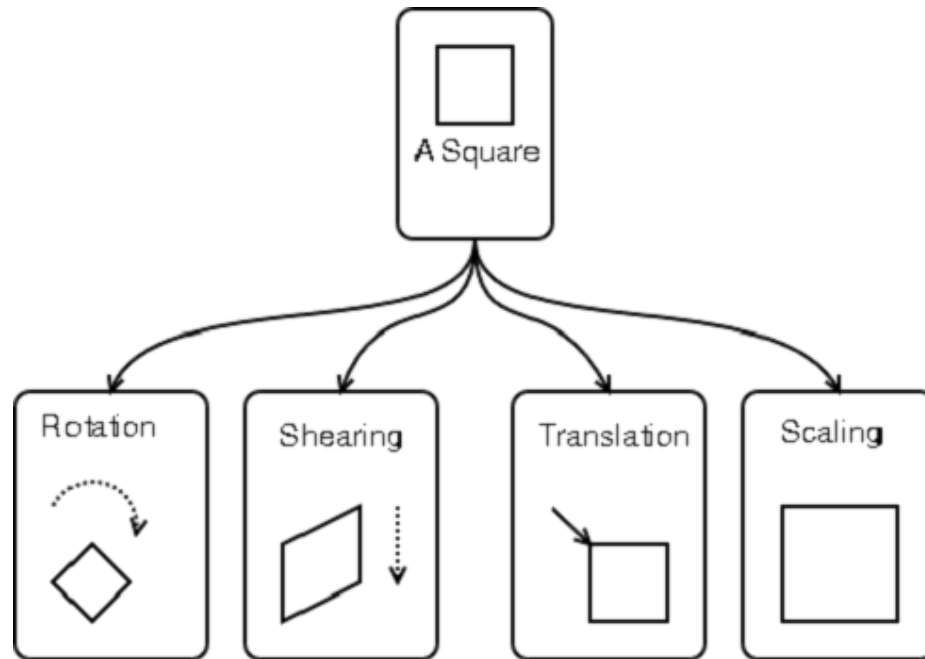
- Intensity interpolation

- Nearest neighbor
- Bilinear interpolation
- Bicubic interpolation



Affine Transform

Affine transform can **scale**, **rotate**, **translate** or **sheer** a set of coordinate points, depending on the value of T .

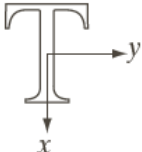
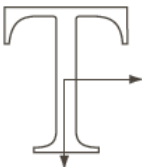
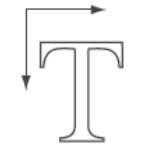


$$[x \quad y \quad 1] = [v \quad w \quad 1]\mathbf{T}$$

$$\text{where } \mathbf{T} = \begin{bmatrix} t_{11} & t_{12} & 0 \\ t_{21} & t_{22} & 0 \\ t_{31} & t_{32} & 1 \end{bmatrix}$$

TABLE 2.2

Affine transformations based on Eq. (2.6.–23).

Transformation Name	Affine Matrix, $\mathbf{T}$	Coordinate Equations	Example
Identity	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x = v$ $y = w$	
Scaling	$\begin{bmatrix} c_x & 0 & 0 \\ 0 & c_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x = c_x v$ $y = c_y w$	
Rotation	$\begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x = v \cos \theta - w \sin \theta$ $y = v \sin \theta + w \cos \theta$	
Translation	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix}$	$x = v + t_x$ $y = w + t_y$	
Shear (vertical)	$\begin{bmatrix} 1 & 0 & 0 \\ s_v & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x = v + s_v w$ $y = w$	
Shear (horizontal)	$\begin{bmatrix} 1 & s_h & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x = v$ $y = s_h v + w$	

$$\begin{bmatrix} x & y & 1 \end{bmatrix} = \begin{bmatrix} v & w & 1 \end{bmatrix} \mathbf{T}$$

$$\text{where } \mathbf{T} = \begin{bmatrix} t_{11} & t_{12} & 0 \\ t_{21} & t_{22} & 0 \\ t_{31} & t_{32} & 1 \end{bmatrix}$$

Two Possible Implementations

Forward Mapping

- Scan the pixels of the input image and compute the spatial location (x, y) in the output image $[x \ y \ 1] = [v \ w \ 1]\mathbf{T}$.
- Two problems
 - Multiple pixels are mapped into the same location
 - Some output locations may not be assigned a pixel

Inverse Mapping

- Scan the output pixel locations and at each location (x, y) compute the corresponding location in the input image using $[v \ w \ 1] = [x \ y \ 1]\mathbf{T}^{-1}$
- Inverse mappings are more efficient in practical applications.

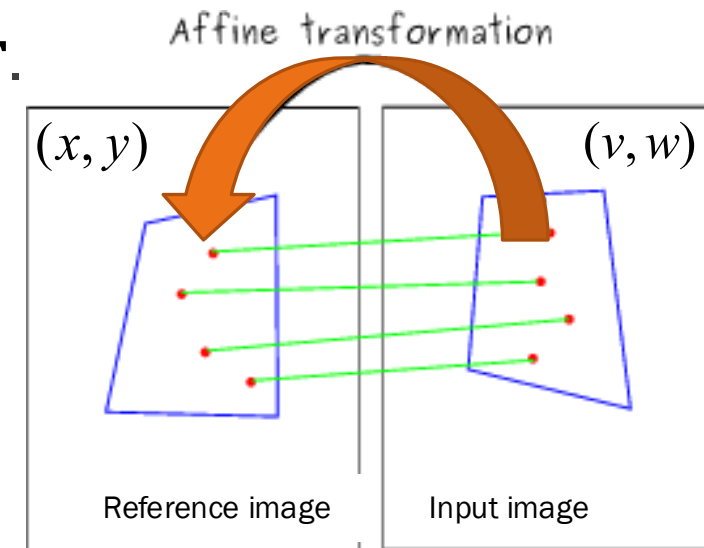




FIGURE 2.36 (a) A 300 dpi image of the letter T. (b) Image rotated 21° clockwise using nearest neighbor interpolation to assign intensity values to the spatially transformed pixels. (c) Image rotated 21° using bilinear interpolation. (d) Image rotated 21° using bicubic interpolation. The enlarged sections show edge detail for the three interpolation approaches.

Finding Geometric Transformation by System of Linear Equations (1)

$$[x \quad y \quad 1] = [v \quad w \quad 1] \begin{bmatrix} t_{11} & t_{12} & 0 \\ t_{21} & t_{22} & 0 \\ t_{31} & t_{32} & 1 \end{bmatrix} \longrightarrow \begin{cases} x = t_{11}v + t_{21}w + t_{31} \\ y = t_{12}v + t_{22}w + t_{32} \end{cases}$$

$$\begin{pmatrix} v & w & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & v & w & 1 \end{pmatrix}_{2 \times 6} \begin{pmatrix} t_{11} \\ t_{21} \\ t_{31} \\ t_{12} \\ t_{22} \\ t_{32} \end{pmatrix}_{6 \times 1} = \begin{pmatrix} x \\ y \end{pmatrix}_{2 \times 1}$$

Each control point pair will provide two equations.

Finding Geometric Transformation by System of Linear Equations (2)

$(v_{1:4}, w_{1:4}) \rightarrow (x_{1:4}, y_{1:4})$
(four pairs of control points)

$$\begin{pmatrix} v_1 & w_1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & v_1 & w_1 & 1 \\ v_2 & w_2 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & v_2 & w_2 & 1 \\ v_3 & w_3 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & v_3 & w_3 & 1 \\ v_4 & w_4 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & v_4 & w_4 & 1 \end{pmatrix} \begin{pmatrix} t_{11} \\ t_{21} \\ t_{31} \\ t_{12} \\ t_{22} \\ t_{32} \end{pmatrix} = \begin{pmatrix} x_1 \\ y_1 \\ x_2 \\ y_2 \\ x_3 \\ y_3 \\ x_4 \\ y_4 \end{pmatrix}$$

$$\mathbf{A} = \begin{pmatrix} v_1 & w_1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & v_1 & w_1 & 1 \\ v_2 & w_2 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & v_2 & w_2 & 1 \\ v_3 & w_3 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & v_3 & w_3 & 1 \\ v_4 & w_4 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & v_4 & w_4 & 1 \end{pmatrix} \quad \mathbf{t} = \begin{pmatrix} t_{11} \\ t_{21} \\ t_{31} \\ t_{12} \\ t_{22} \\ t_{32} \end{pmatrix} \quad \mathbf{B} = \begin{pmatrix} x_1 \\ y_1 \\ x_2 \\ y_2 \\ x_3 \\ y_3 \\ x_4 \\ y_4 \end{pmatrix}$$

$$\mathbf{A}\mathbf{t} = \mathbf{B} \rightarrow \begin{aligned} \mathbf{t} &= \text{linsolve}(\mathbf{A}, \mathbf{B}) \\ \mathbf{t} &= \text{np.linalg.solve}(\mathbf{A}, \mathbf{B}) \end{aligned}$$

Image Registration Steps

Step 1: Select (Data Cursor) at least 3 pairs of **control points** from the image pair $I(x, y) \leftrightarrow J(v, w)$.

Step 2: Write a system of linear equations based on the control points, and solve the system of linear equations (linsolve or np.linalg.solve)

$$\mathbf{t} = [t_{11} \quad t_{21} \quad t_{31} \quad t_{12} \quad t_{22} \quad t_{31}]$$
$$\mathbf{T} = \begin{bmatrix} t_{11} & t_{12} & 0 \\ t_{21} & t_{22} & 0 \\ t_{31} & t_{31} & 1 \end{bmatrix}$$

Step 4: Reshape $\mathbf{t}$ into the transformation matrix $\mathbf{T}$

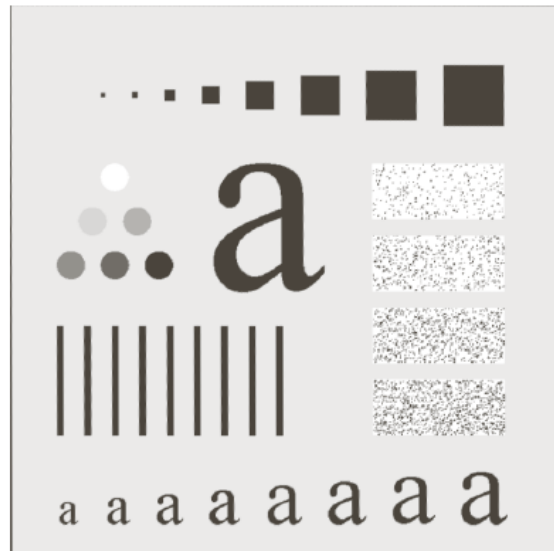
Step 5: For each location in $I(x, y)$, use $\mathbf{T}^{-1}$ to find the corresponding location in $J(v, w)$ and interpolate its value.



Reference
image $I(x, y)$



Input image $J(v, w)$



a	b
c	d

FIGURE 2.37

Image registration.

(a) Reference image. (b) Input (geometrically distorted image). Corresponding tie points are shown as small white squares near the corners.

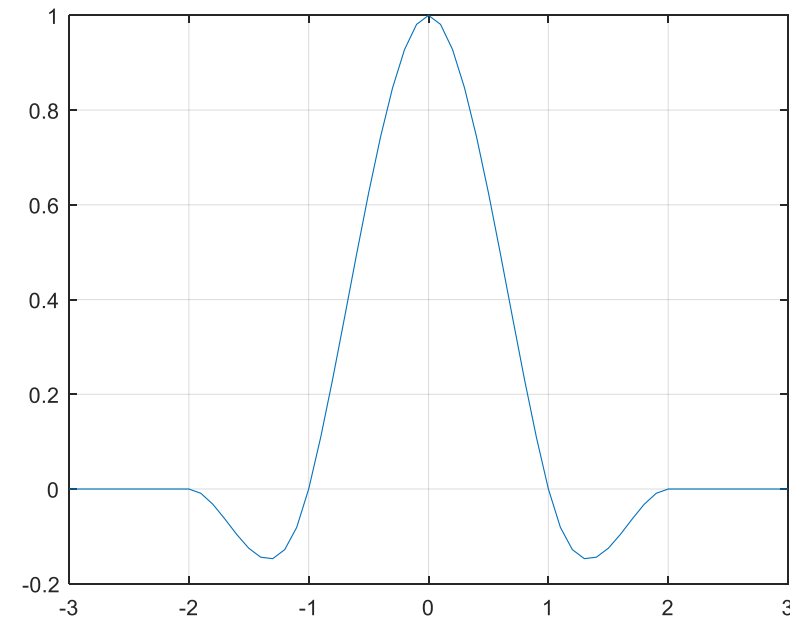
(c) Registered image (note the errors in the borders).

(d) Difference between (a) and (c), showing more registration errors.

Matlab Programming Tip

The Matlab function of the interpolation kernel (Ricker.m) is provided that can be further extended to array computation.

- `clear all;`
- `X=-3:0.1:3;`
- `Y=arrayfun(@Ricker,X);`
- `plot(X,Y); grid on;`

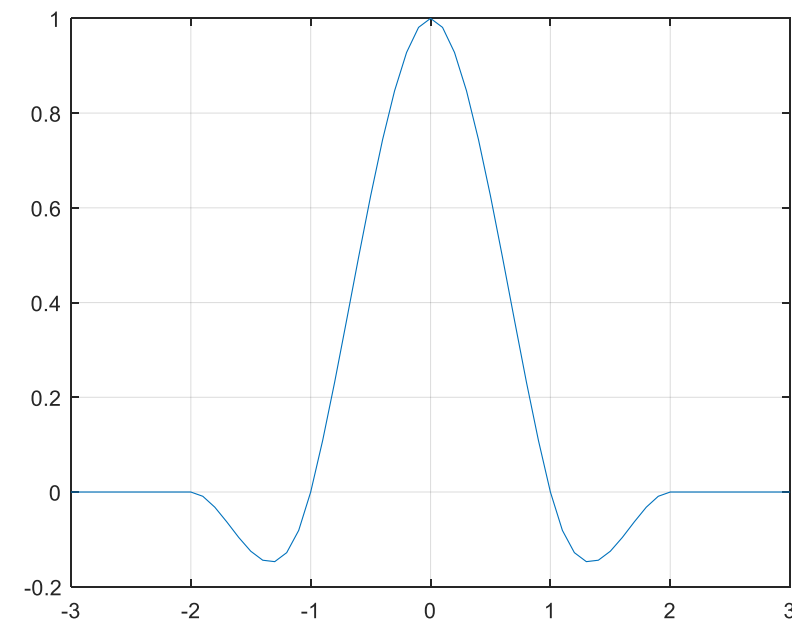


Programming Tip

The function of the interpolation kernel (ricker.m, ricker.py) is provided that can be further extended to array computation.

- clear **all**;
- $X = -3:0.1:3$;
- $Y = \text{arrayfun}(@\text{ricker}, X)$;
- $\text{plot}(X, Y)$; grid **on**;

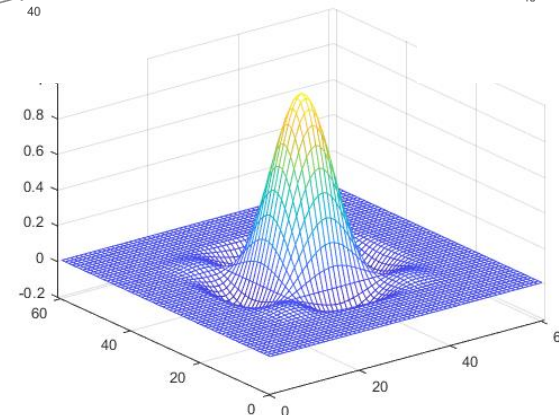
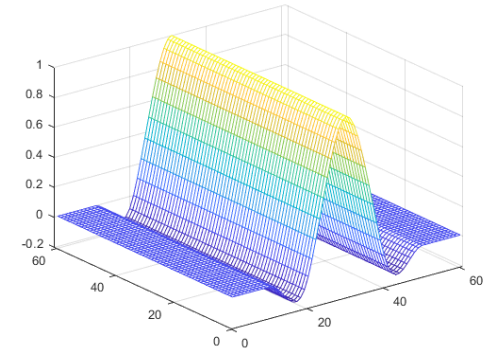
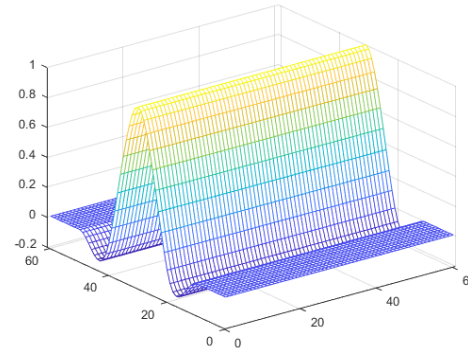
```
1  ✓ import numpy as np
2  import matplotlib.pyplot as plt
3  from ricker import myRicker
4
5  # X = np.arange(-3,3,61)
6  X = np.arange(-3, 3.0001, 0.1)
7  Y = myRicker(X)
8
9  plt.plot(X,Y)
10 plt.show()
```



Matlab Programming Tip

The Matlab function of the interpolation kernel (ricker.m) is provided that can be further extended to array computation.

- `x=-3:0.1:3;`
- `y=-3:0.1:3;`
- `[X,Y]=ndgrid(x,y);`
- `Z1=arrayfun(@ricker,X);`
- `Z2=arrayfun(@ricker,Y);`
- `Z3=Z1.*Z1;`
- `figure(1),mesh(Z1);`
- `figure(2),mesh(Z2);`
- `figure(3),mesh(Z3);`



Python Programming Tip

```
12 x = np.arange(-3, 3.0001, 0.1)
13 y = np.arange(-3, 3.0001, 0.1)
14
15 X,Y = np.meshgrid(x,y, indexing='ij')
16 Z1 = myRicker(X)
17 Z2 = myRicker(Y)
18 Z3 = Z1*Z2
19
20 fig = plt.figure(figsize=(12, 4))
21
22 ax1 = fig.add_subplot(131, projection='3d')
23 ax2 = fig.add_subplot(132, projection='3d')
24 ax3 = fig.add_subplot(133, projection='3d')
25
26 ax1.plot_surface(X, Y, Z1)
27 ax1.set_title("Z1")
28
29 ax2.plot_surface(X, Y, Z2)
30 ax2.set_title("Z2")
31
32 ax3.plot_surface(X, Y, Z3)
33 ax3.set_title("Z3")
34
35 plt.tight_layout()
36 plt.show()
```

